Cloud Workload Forecasting & Analysis

Reproducible short-horizon forecasts from the curated Azure VM trace

Executive summary

This project constructs the two required aggregate targets from 1,340,000 VM-level CPU readings: distinct active VMs and mean VM CPU utilization. It compares persistence, a six-reading moving average, and a 24-reading linear trend using chronological rolling-origin validation. Persistence is selected for both targets; the active-VM target is constant in this curated complete-trace population, while persistence is the strongest CPU baseline among the tested methods.

Dataset and split

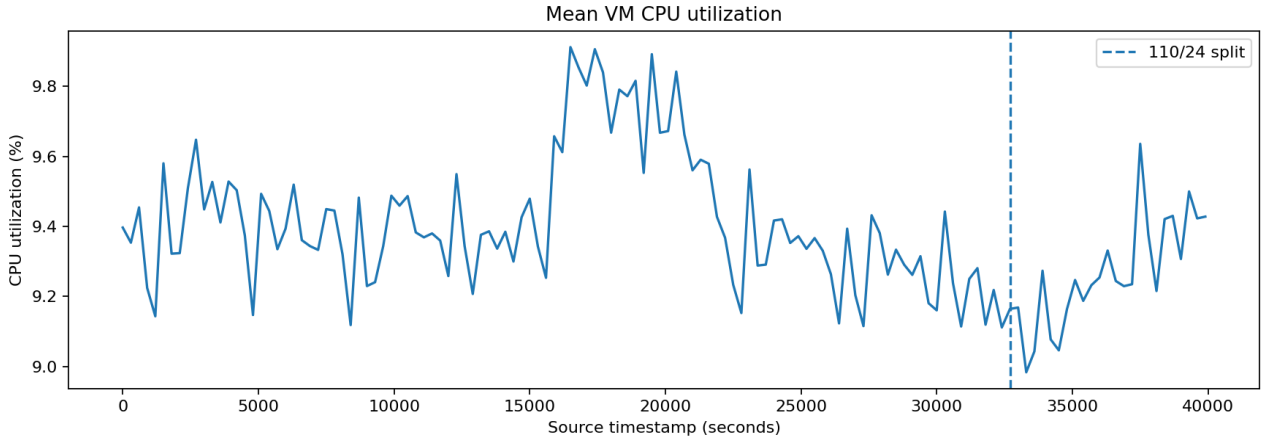
The data-v1 subset contains 10,000 VMs with 134 readings per VM at five-minute cadence, covering source timestamps 0–39,900 seconds. The first 110 aggregate timestamps are the modeling window; the final 24 timestamps are the required two-hour holdout. Source timestamps are elapsed seconds, not wall-clock time.

Quality check	Observed result
CPU reading rows	1,340,000
Valid avg_cpu rows	1,340,000
Unique VMs	10,000
Aggregate timestamps	134
Duplicate VM/timestamp rows	0
Non-300-second gaps	0
Missing grid timestamps	0
Metadata missing cells	0

Key caveat: the dataset was curated to include only VMs with complete traces. Therefore, active_vm_count is exactly 10,000 at every timestamp. This task does not provide a meaningful test of VM arrivals, deletions, or autoscaling churn.

1. Exploratory analysis

Aggregate CPU utilization averages 9.388% across all timestamps, with a standard deviation of 0.195 percentage points and a range of 8.984%–9.912%. In the first 110 timestamps, the range is 9.112%–9.912%. The short trace shows local level changes, not enough repeated cycles to establish daily or weekly seasonality.



VM-level heterogeneity

The metadata population comprises 6,180 delay-insensitive VMs, 3,091 interactive VMs, and 729 unknown-category VMs. Segment summaries below are computed across per-VM trace means, so each VM contributes one value.

VM category	VMs	Mean (%)	Median (%)	P95 (%)
Delay-insensitive	6,180	8.236	5.616	25.747
Interactive	3,091	11.493	5.565	46.025
Unknown	729	10.233	3.894	48.393

Interactive VMs have a higher mean trace utilization and a wider upper tail in this sample. These are descriptive associations, not causal explanations for timestamp-level changes. The aggregate mean is unweighted by VM core count and suppresses individual VM extremes.

2. Forecasting and validation

Three transparent candidates were evaluated independently for each target: (1) persistence repeats the last value, (2) moving average repeats the mean of the latest six observations (30 minutes), and (3) linear trend fits a line to the latest 24 observations and extrapolates 24 steps. No calendar features were introduced because the source has no wall-clock semantics and the observation window is short.

Validation protocol

The first 110 points were evaluated using three rolling-origin windows with origins at indices 62, 74, and 86. Each validation forecast predicts the next 24 points using only history available before that origin. MAE and RMSE are averaged across windows. CPU errors are percentage points, not relative percent error.

CPU model	Mean window MAE	Mean window RMSE	Normalized MAE	Relative MAE vs persistence
persistence	0.1848	0.2174	0.231	baseline
moving_average_6	0.2491	0.2790	0.311	-34.8%
linear_trend_24	0.4339	0.4811	0.542	-134.8%

Persistence achieves CPU MAE 0.1848 percentage points and RMSE 0.2174. The moving average has MAE 0.2491 and the linear trend 0.4339; neither improves on persistence. Smoothing lags level changes, while a fitted short-term slope compounds error over a two-hour horizon.

For active_vm_count, persistence and moving average both achieve zero validation error because the target is constant. Linear trend has only negligible floating-point residuals; persistence is selected explicitly for a constant series. Relative improvement against a zero-error baseline is not meaningful.

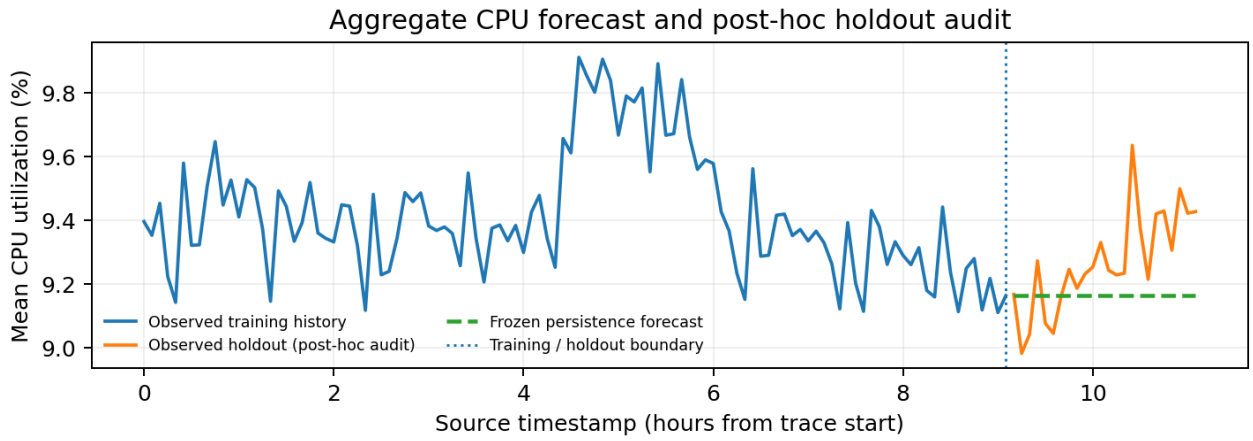
3. Error analysis and frozen holdout audit

Peak-period check

At each validation origin, high load was defined as a value above the 90th percentile of that origin's training history. Only six validation points met this criterion, all in the earliest window. Persistence MAE on these six points was 0.073 percentage points, versus 0.136 for linear trend. This is too small a sample to support a broad claim about peak prediction.

Post-hoc holdout audit

After predictions were frozen, we compared them with the public final 24 observations for diagnostic reporting only. These values were not used for model selection or tuning. The holdout CPU MAE is 0.1468 percentage points and RMSE is 0.1848; mean forecast-minus-actual bias is -0.1044 percentage points. The forecast is constant at 9.1646%, equal to the final training observation.



The holdout actual mean is 9.269%, above the 9.1646% forecast; the frozen forecast under-forecasts on average in this period. A single two-hour period is not proof of generalization and should not be used to retune the model.

4. Operational recommendation

Treat this aggregate forecast as a short-horizon monitoring signal, not a standalone autoscaling decision. Refresh it every five minutes; compare it with service-specific warning thresholds and recent observed trends; calibrate a guard band from under-forecast residuals; and monitor actual-versus-forecast residuals for drift. The 90th percentile of positive persistence under-forecast residuals in the validation windows is about 0.108 percentage points, but it is based on only nine positive residuals and is not a production guarantee.

Capacity-planning interpretation

The target is the arithmetic mean of VM-level CPU percentages. It is not total CPU demand and cannot directly determine the number of cores or hosts. Production capacity planning should add capacity-weighted utilization, segment-level forecasts, per-service saturation signals, and explicit scaling constraints. Under-provisioning can increase latency and failures; over-provisioning wastes capacity, so margins must reflect workload criticality.

Limitations

• Only about 11.1 hours of data are available; daily/weekly seasonality cannot be learned. • Three rolling-origin windows provide a noisy estimate for a 24-step horizon. • Complete-trace curation makes active_vm_count constant and prevents a churn forecast evaluation. • Aggregate mean CPU hides per-VM peaks and is not capacity-weighted. • The public holdout audit is post-hoc and does not replace an unseen private evaluation set.

Reproduction

From the repository root, install requirements, place the official data-v1 files in challenge_data/, and run:

```
python -m src.eda --data challenge_data --output reports/eda
python -m src.predict --data challenge_data --output predictions.csv
pytest -q
python scripts/build_report.py
```

The submission file has exactly 24 chronological rows and the columns timestamp, active_vm_count, aggregate_cpu_utilization. The raw challenge dataset is excluded from the project archive. Six automated tests cover aggregation, baselines, holdout isolation, output contract, and a small reviewer smoke dataset.